MQTT-Discovery and UPnP: a comparison

Daniele Foschi
Alma Mater Studiorum Università di Bologna UNIBO
Bologna, Italy
daniele.foschi2@studio.unibo.it
https://github.com/DaniDF/MQTT-Discovery-vs-UPnP

***Abstract*—TODO**

***Index Terms*—MQTT, discovery, upnp, project activity.**

I. Introduction

Internet of Things (IoT) has a lot of definitions: from International Telecommunication Union is *"A global infrastructure for the information society, enabling advanced services by interconnecting (physical and virtual) things based on existing and evolving interoperable information and communication technologies"*. IEEE defines IoT as a list of features:

- A network
- Objects are identifiable as things with unique identifiers
- When a thing communicate should use internet at least in one segment of the communication
- Things can have sensing or actuating or both capabilities
- They should be programmable as result of injection of intelligence
- The offered services should be available anywhere, anytime and for anything

In a environment populated by a large variety of different devices with different needs and capabilities designing a standard useful for communicating with them is a challenge. Discovery protocols are designed to reconcile the dichotomy between the requirement for highly specific, ad-hoc protocols capable of granular device control and the necessity for generalized frameworks that can accommodate a highly heterogeneous ecosystem of devices. By employing these protocols, an IoT device achieves autonomous configuration through active participation in the advertisement and solicitation of network-based services. Devices functioning as service providers systematically broadcast their available resources, the specific services they offer, and the requisite operational guidelines for invocation, such as the appropriate Application Programming Interfaces (APIs). Furthermore, discovery protocols streamline the connectivity paradigm between clients and target resources or services. Consequently, this architecture ensures that clients are adequately equipped to transmit necessary input parameters to the services and successfully retrieve the corresponding computational results.

In particular this dissertation is focused on Universal Plug and Play (UPnP) and a protocol based on Message Queuing Telemetry Transport (MQTT) knwon as MQTT Discovery (MQTT-D). UPnP is a discovery protocol without infrastructural nodes where each node can advertise and listen for services. In particular nodes that make available services are called *devices* and those searching for services are called *control-points*. In MQTT-D there is a third entity called *broker* that facilitates the communication between services and clients.

This dissertation wants to highlight the performance differences between UPnP and MQTT-D in IoT environment. The main focus is measuring and comparing latency and robustness in different deployment configurations: how different amount of data producers and consumers affect the protocol performances.

II. Discovery protocols for IoT

Resource discovery protocols can be categorised into: centralised and distributed. In centralised approaches such as MQTT, resources are registered in a directory that serves as intermediary between providers and users. In distributed approaches such as UPnP, users discover active resources through direct communication with resources in a Peer-to-Peer (P2P) fashion with no need for an intermediary directory [1].

UPnP is a P2P discovery service without hierarchical structuring. In UPnP two entities can be distinguished: *control-point* the entity that look for services and resources and *device* the entity that make services available. A device can dynamically join a network and obtain a locally valid *IP* address. After it can make its capabilities visible and discover others, if present, and dynamically leave the network. UPnP uses *XML* for the description of the resources, services and action invocations.

<table>
<tr><td colspan="4">UPnP vendor [purple-italic]</td></tr>
<tr><td colspan="4">UPnP Forum [red-italic]</td></tr>
<tr><td colspan="4">UPnP Device Architecture [green-bold]</td></tr>
<tr><td rowspan="2">SSDP [blue]</td><td rowspan="2">Multicast events [navy-bold]</td><td>SOAP [blue]</td><td>GENA [navy-bold]</td></tr>
<tr><td>HTTP [black]</td><td>HTTP [black]</td></tr>
<tr><td colspan="2">UDP [black]</td><td colspan="2">TCP [black]</td></tr>
<tr><td colspan="4">IP [black]</td></tr>
</table>

Fig. 1. UPnP Protocol Stack [2]

UPnP can be divided into 4 parts: Simple Service Discovery Protocol (SSDP), HyperText Transfer Protocol (HTTP),

Simple Object Access Protocol (SOAP) and General Event Notification Architecture (GENA). SSDP is a network protocol based on the Internet protocol suite for advertisement and discovery of network services and presence information. It is a text-based protocol based on a non fully compliant version of HTTP over UDP [2]. SOAP is a messaging protocol specification for exchanging structured information in XML message format. In UPnP it is used by the *control-point* for requesting the execution of services with specified arguments on the *device* and by the *device* for returning the results. GENA is an event notification protocol used by *devices* to publish variables updates to subscribed *control-points*. GENA supports both unicast and multicast event notification.

MQTT-D is a discovery protocol based on MQTT. MQTT is a lightweight publish-subscribe messaging protocol designed for constrained devices. MQTT-D is a metadata-exchange convention built on top of the MQTT protocol that standardises how devices and services describe themselves and publish their configuration to an MQTT broker. It enables automatic detection, registration, and integration of devices in home-automation or IoT ecosystems without requiring manual configuration.

This dissertation aims to evaluate the performance of MQTT-D and UPnP in deployments with different number of deployed devices and different number of control devices. For MQTT-D will be evaluated how the performances changes with respect to the requested Quality Of Service (QOS). The evaluated metrics for this study are latency and robustness. The first wants to measure the time needed for discovery and the execution of the first interaction between *control-point* and *device*. The second wants to measure the ratio between the discovered devices and the actual deployed devices.

III. Implementation

In order to implement a testing environment a fully compatible UPnP *device* and *control-point* were implemented based on the official documentation [2]. All its parts were implemented: SSDP, SOAP, GENA. For HTTP the standard library already available in Go Lang is used. In order to demonstrate compatibility based on the standard a third part UPnP client (only) library is used [3].

For MQTT-D a already available implementation of MQTT is used and was implemented only a wrapper for the MQTT-D data format. As MQTT-Broker Mosquitto is chosen as one of the most popular broker in IoT deployments.

All the software components implemented by the author are written in Go Lang for maximising efficiency but still using a higher level programming language.

IV. Experimental results

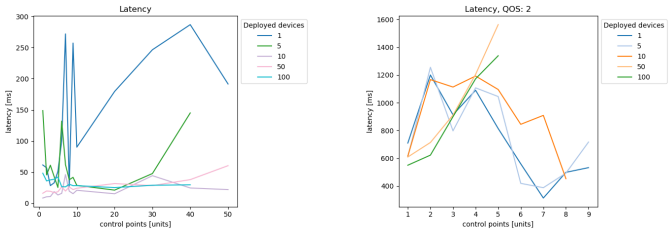
The experiments were conducted in a real closed network where only the testing equipment were connected. All the experiments were run both wireless using IEEE802.11n and cabled. During the tests were deployed 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 devices and control-points in all the possible values combinations. In case of MQTT each combination were tested for QOS 0, 1 and 2. Also each combination were tested 30 times and the results are averaged in order to minimise statistical fluctuations.

In SSDP every time a node wants to discover a *device* sends a M-SEARCH request asking if a device is present with some described characteristics. As soon as a *device* receives the request and confirms that it is compliant with the requested device it sends back a response. Since the number of responses can not be predicted the *control-point* add to the search request the parameter MX. This parameter indicates the maximum value of seconds that the *control-point* waits for the response. A *device* that need to respond to a request has to wait a random number of milliseconds between 0 and the value specified by MX. In this way a possible Denial-of-Service (DOS) done through the flooding of the network of responses is mitigated [2].

The experiments tried to evaluate the effectiveness of the MX parameter by removing the random wait time from the *device* response.

In the following subsections present the measured metrics for values 1, 5, 10, 50, 100 but a more detailed verision can be find in appendix A.

A. Lantency comparison



(a) (b)

Fig. 2. Comparison of (a) UPnP and (b) MQTT latency. See 10 and 4, 5, 6

- Osservazione

Grafico 1 5 10 50 100 comparison M

- Osservazione

1) Wired vs Cabled:

- Motivazione dei test

Grafico 1 10 50 comparison UvM

- Osservazione

B. Robustness comparison

- Espandere obiettivo test (motivazione)
- Descrizione esperimento (metodi)

Grafico 1 5 10 50 100 comparison U

- Osservazione

Grafico 1 5 10 50 100 comparison M

- Osservazione

1) Wired vs Cabled:

- Motivazione dei test

Grafico 1 10 50 comparison UvM

- Osservazione

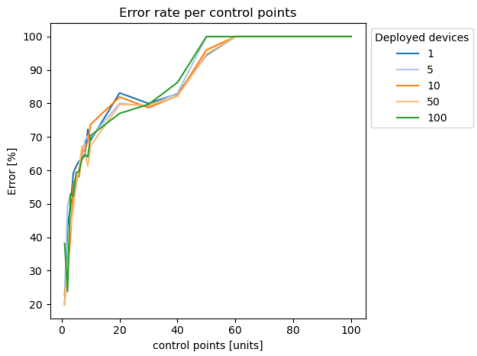


Fig. 3. Simulation results for the network.

V. Conclusions

- Cosa è statto fatto in breve
- Cosa è stato dimostrato
- Future directions

Appendix

A. MQTT

1) MQTT latency:

2) MQTT robustness:

B. UPnP

1) UPnP latency:

2) UPnP robustness:

References

[1] K. Khalil, K. Elgazzar, and M. Bayoumi, "A comparative analysis on resource discovery protocols for the internet of things," in *2018 IEEE Global Communications Conference (GLOBECOM)*, 2018, pp. 1–7.

[2] O. C. Foundation, "Upnp device architecture 2.0," Open Connectivity Foundation, Tech. Rep., April 2020, document Revision Date: April 17, 2020. [Online]. Available: https://openconnectivity.org/upnp-specs/UPnP-arch-DeviceArchitecture-v2.0-20200417.pdf

[3] Huin, "goupnp," https://github.com/huin/goupnp, 2026, gitHub repository.

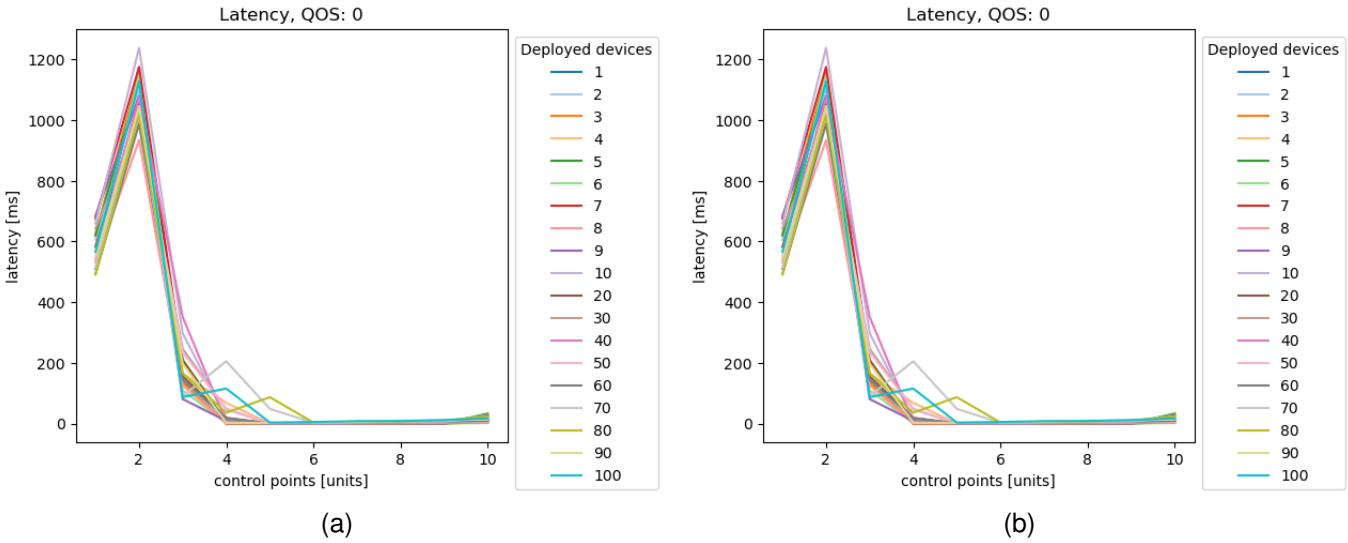


Fig. 4. MQTT latency QOS 0

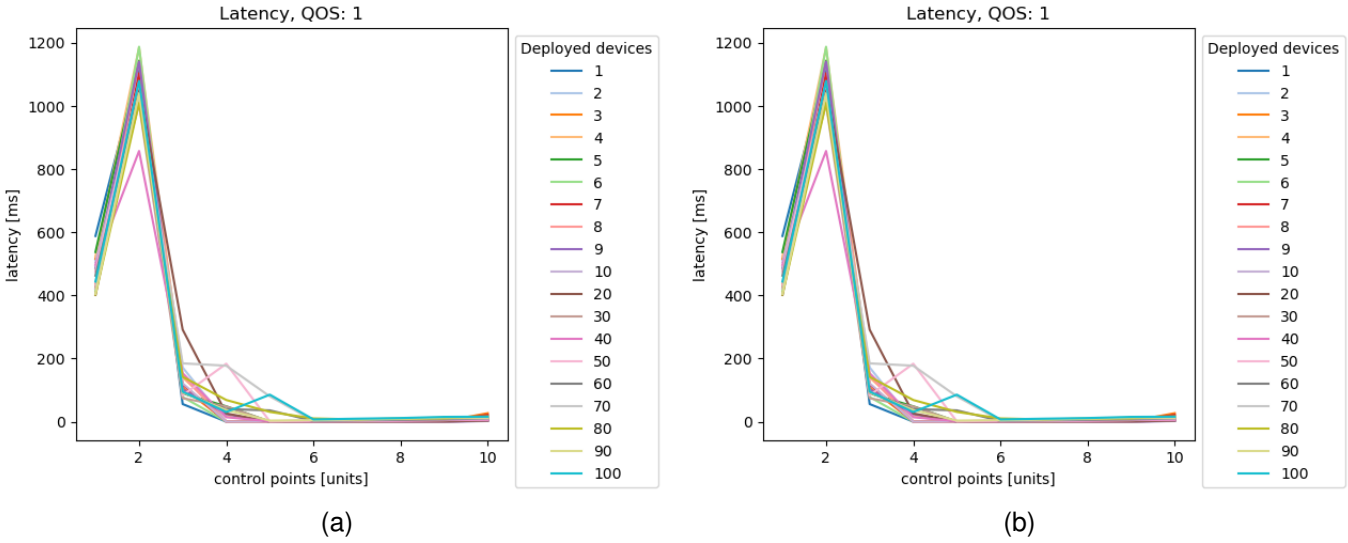


Fig. 5. MQTT latency QOS 1

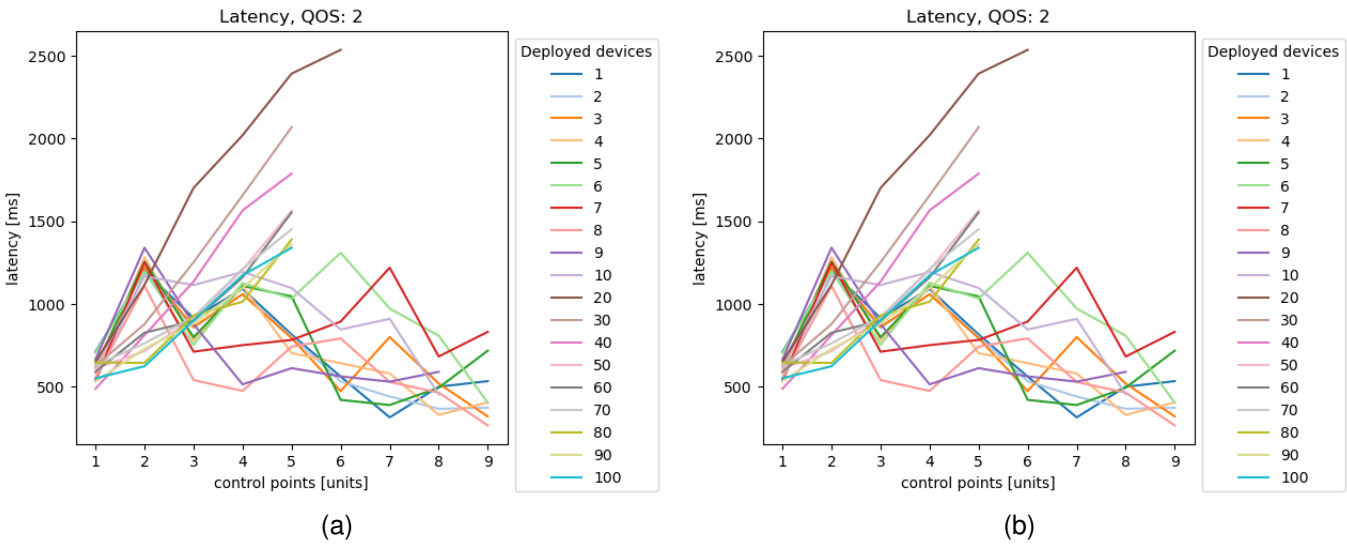


Fig. 6. MQTT latency QOS 2

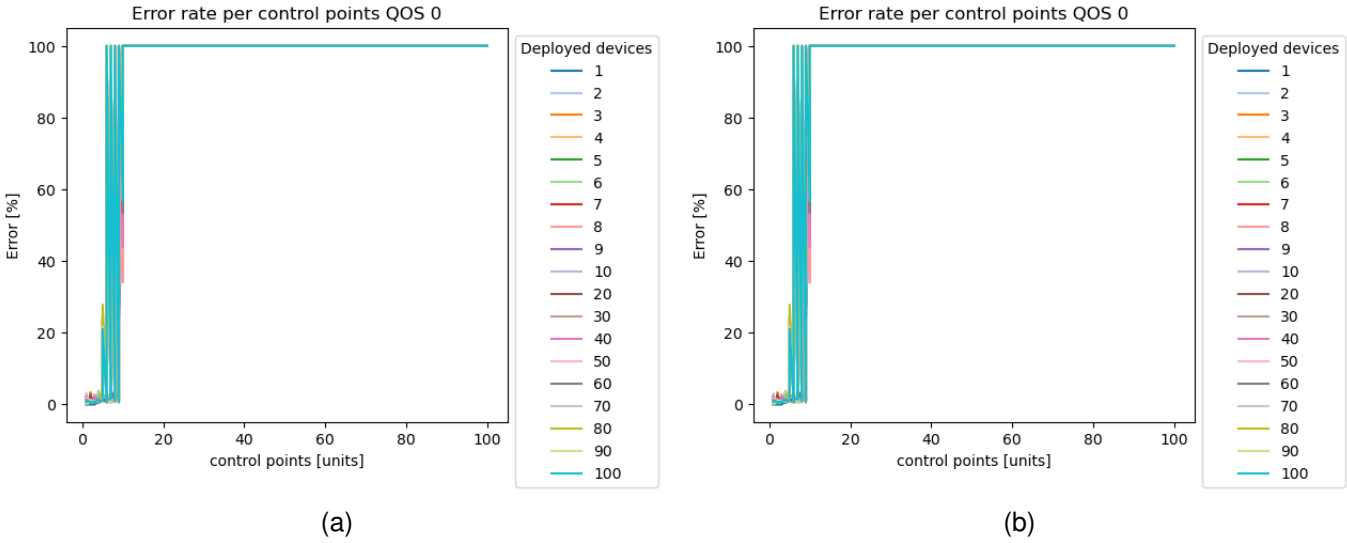


Fig. 7. MQTT latency QOS 0

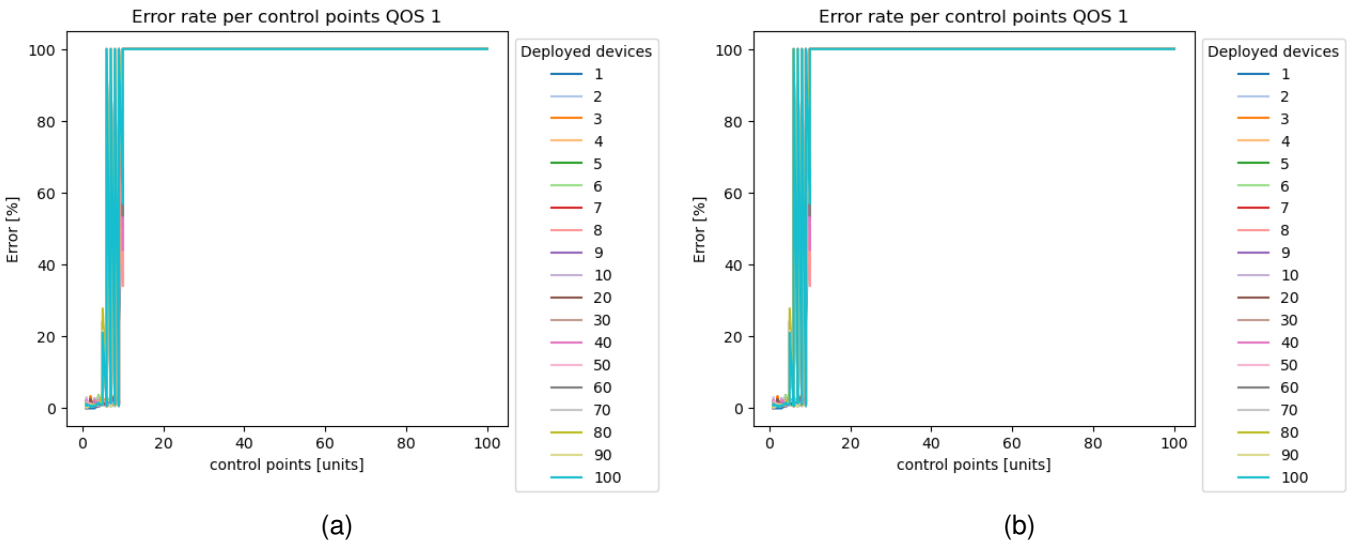


Fig. 8. MQTT latency QOS 1

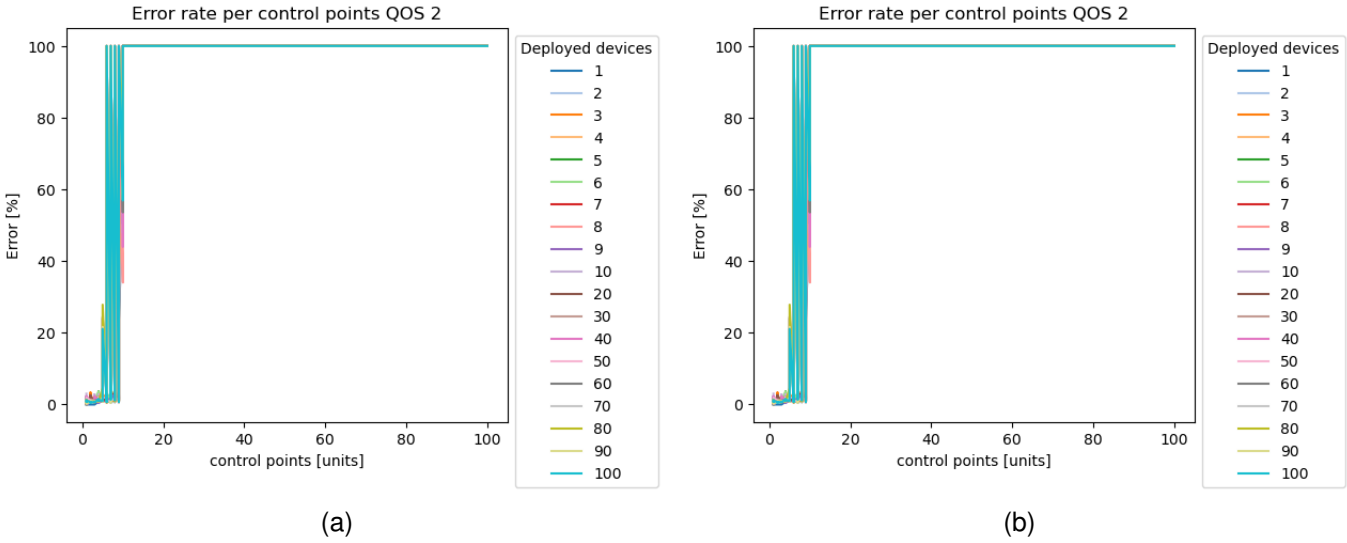


Fig. 9. MQTT latency QOS 2

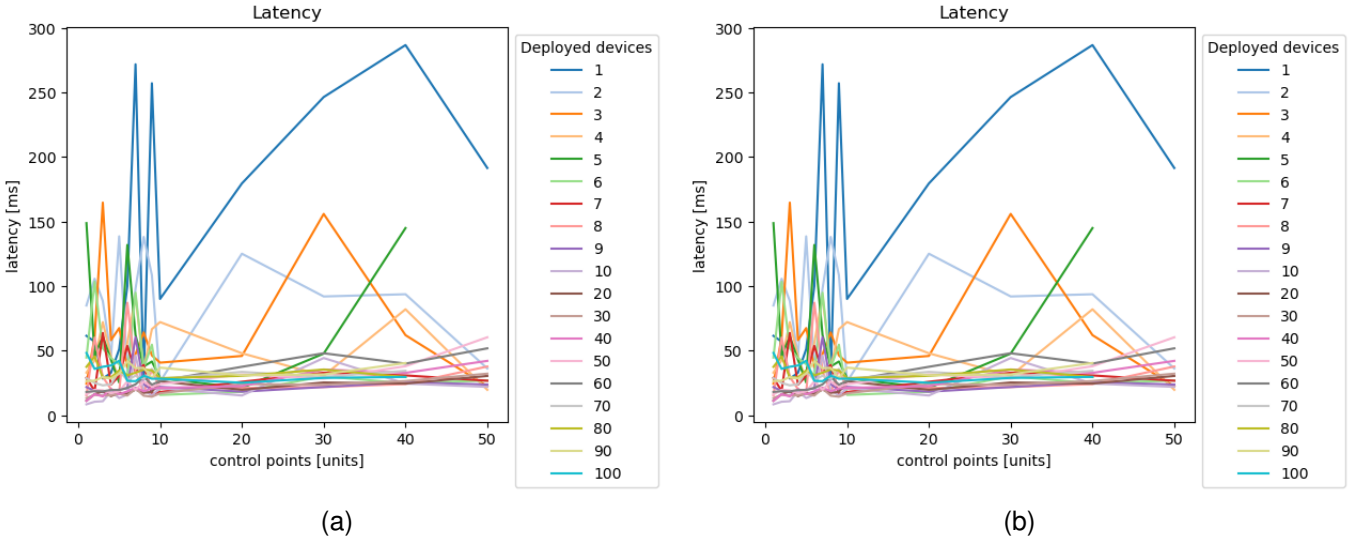


Fig. 10. UPnP latency

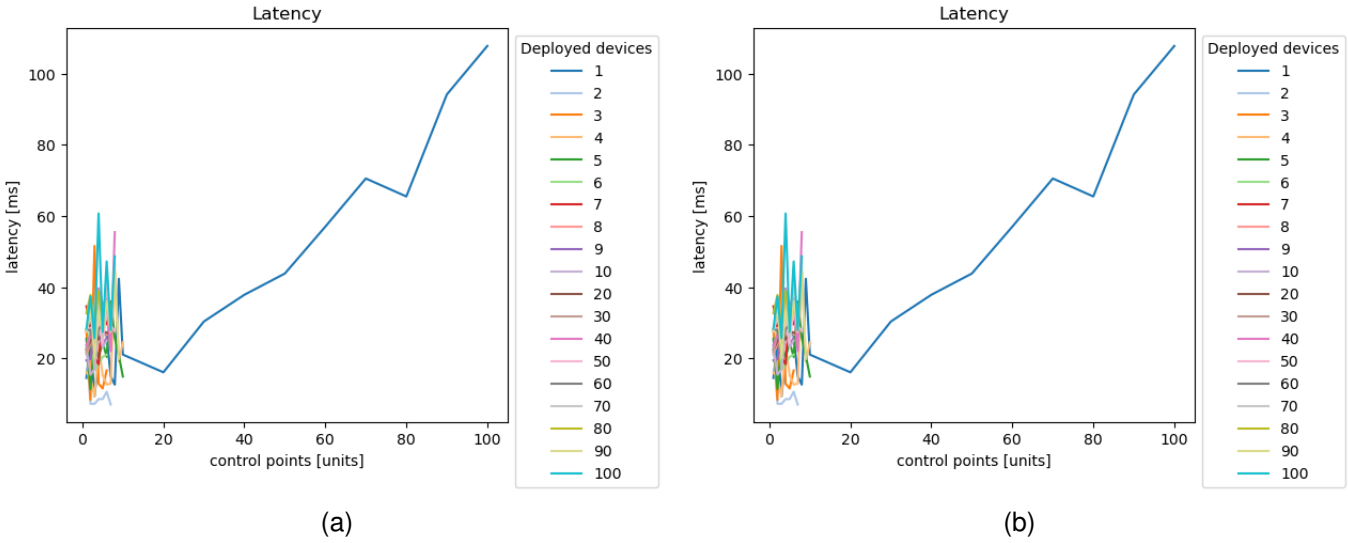


Fig. 11. UPnP latency with no random delay

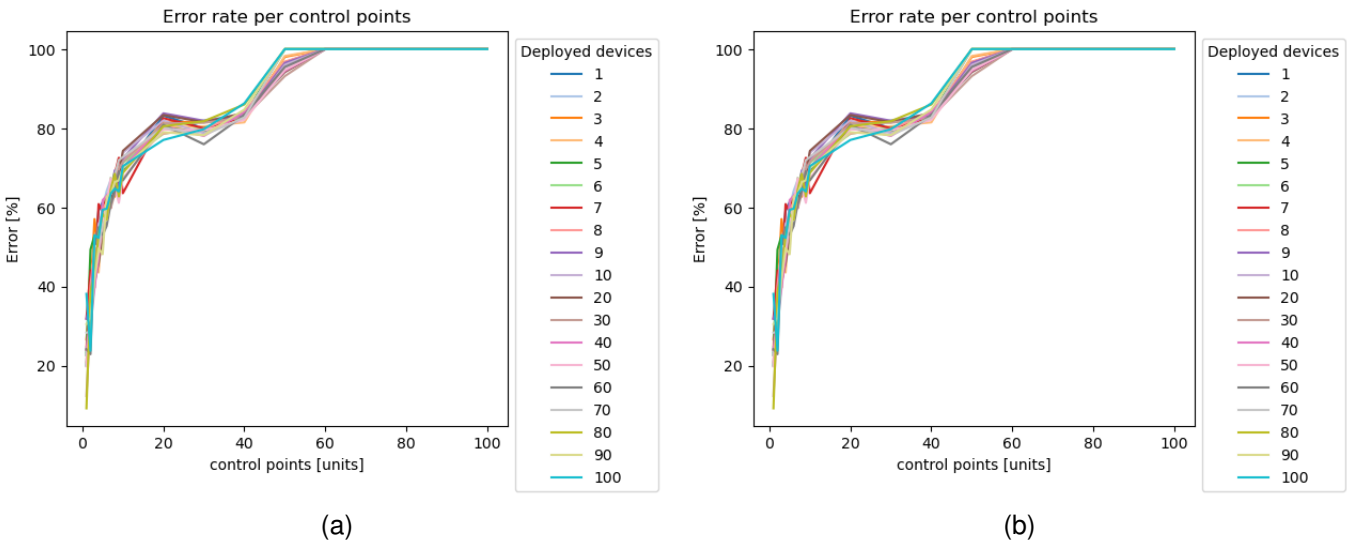


Fig. 12. UPnP robustness

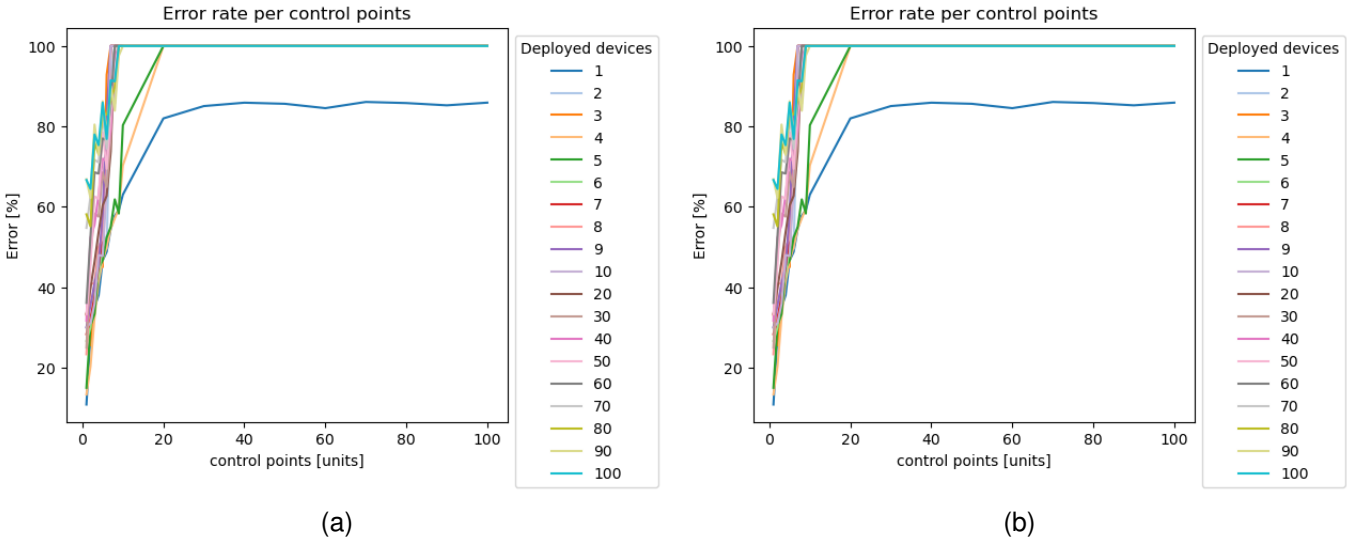


Fig. 13. UPnP robustness with no random delay